

6. Инструкция по самостоятельному развёртыванию

Три варианта:

- **А. Полный стенд на Linux-сервере** — как `thinkfaster.ru`: вся система с интерфейсом.
- **Б. Локальный стенд потока** — эмулятор → воронка → модель → аудит на своей машине, без интерфейса.
- **В. Обновление уже развёрнутого стенда** из GitHub.

Команды взяты из рабочих выкаток dev и prod и из workflow репозиториев. Особенности, из-за которых что-то может не получиться, разобраны в [SYSTEM.md](#), [Часть VI](#). Пошаговая инструкция по инфраструктуре для человека без опыта — [think-infra: QUICK_START.md](#).

6.1. Что нужно заранее

Что	Для А	Для Б
Linux-сервер (Ubuntu 22.04+ или Debian 12+), Docker 24+, Docker Compose v2, git, openssl	да	да (или WSL2 / Docker Desktop)
Ресурсы	оценка: 4 CPU, 8–12 ГБ RAM, 60 ГБ диска (модель — до 2,5 ГБ, Kafka и архив воронки растут)	4 CPU, 8 ГБ RAM
Домен с А-записью на сервер, открытые 80 и 443	да	нет
Доступ к репозиториям: think-infra, think-auth, think-bff, think-front, Think-Faster	да	Think-Faster, think-infra
Пакет модели (архив <code>model-*.tar.gz</code> из закрытого релиза dev-assets Think-Faster) или пересборка из данных исследования	да	да
Эмулятор (закрытый think-test) или своя шина	по желанию	да
Менеджер паролей для ключей Vault	да	нет

Порядок запуска и зависимости — [SYSTEM.md 4.2](#).

6.2. Вариант А: полный стенд

А1. Инфраструктура

```
git clone https://github.com/Think-Faster/think-infra.git && cd think-infra && git checkout prod
```

Заполнить `stands/prod.env` (или создать свой `stands/<стенд>.env` по образцу): `TF_INFRA_DIR`, `DOMAIN`, `LETSencrypt_EMAIL`, `WEB_BIND`. Файл описан в [SYSTEM.md](#), [приложение С](#). Затем:

```
scripts/bootstrap-stand.sh prod
```

Скрипт поднимает и инициализирует Vault — **ключи распечатывания и root-токен показываются один раз**, сохраните их. Затем создаёт политики и роли, генерирует секреты, выкатывает PostgreSQL, Redis, Kafka, RabbitMQ и nginx с сертификатом Let's Encrypt, ставит бэкапы в cron, выдаёт пару CI и личный токен. Подробно — [README.md think-infra](#).

A2. Креды сервисов

Под root-токеном ([SYSTEM.md 4.6](#)):

```
hashicorp/scripts/setup.sh apply
```

Затем выполнить по разу для каждого сервиса: `tf-auth`, `tf-bff`, `tf-funnel`, `tf-model`, `tf-audit`. Сохраните `VAULT_ROLE_ID` и `VAULT_SECRET_ID` каждого:

```
hashicorp/scripts/setup.sh service-credentials tf-auth
```

Секреты, которые вводятся руками, — под личным токеном: SMTP (`TF_MAIL_SMTP_HOST/PORT/USER/PASSWORD`), токен бота (`TF_TG_BOT_TOKEN`), `TF_FUNNEL_SERVICE_SUBS`:

```
scripts/secrets.sh set TF_MAIL_SMTP_HOST
```

A3. tf-auth

Код — [think-auth@prod](#), выкатка — [deploy/docker-compose.yml](#).

```
git clone -b prod https://github.com/Think-Faster/think-auth.git && cd think-auth/deploy && cp .env.example .env
```

```
read -rsp 'VAULT_ROLE_ID: ' R; echo; read -rsp 'VAULT_SECRET_ID: ' S; echo; (umask 077; printf 'VAULT_ROLE_ID=%s\nVAULT_SECRET_ID=%s\n' "$R" "$S" > .vault.env); unset R S
```

```
docker compose --env-file .env --env-file .vault.env build && docker compose --env-file .env --env-file .vault.env up -d; rm -f .vault.env
```

Проверка: `docker ps` → `tf-auth (healthy)`. Первый администратор: `POST /api/auth/register`, затем `SuperUser = true` SQL-запросом ([SYSTEM.md](#), [tf-auth §9](#)).

A4. tf-bff

Код — [think-bff@prod](#), каталог `BFF/deploy`. В `.env` задаются `AUTH_SERVICE_URL=http://tf-auth:8080`, `AUTH_REFRESH_PATH=/refresh`, `AUTH_JWKS_URL=http://tf-auth:8080/.well-known/jwks`, `AUTH_ISSUER=auth-service`, `AUTH_AUDIENCE=api`, `CORS_ALLOWED_ORIGINS=https://<домен>`.

```
git clone -b prod https://github.com/Think-Faster/think-bff.git && cd think-bff/BFF/deploy && cp .env.example .env && cp migration/.env.example migration/.env
```

Миграции и сидинг (группа `admins`, ресурсы): пара Vault передаётся так же, как в A3.

```
docker compose -f migration/docker-compose.migrations.yml --profile migrations up --abort-on-container-exit --exit-code-from bff-migrations
```

```
docker compose build && docker compose up -d
```

Демонстрационные данные (объекты, датчики, люди, группы, график) — по желанию:

```
psql "<строка подключения bff_admin>" -f <Think-Faster>/docs/backend/seed/seed.sql
```

Описание данных — [домены-и-сущности.md §13](#).

A5. tf-front

```
git clone -b prod https://github.com/Think-Faster/think-front.git && cd think-front/deploy && docker compose build && docker compose up -d
```

Контейнер `tf-front` в сети `think-fast-net`, nginx отдаёт его по `/`.

A6. tf-funnel

Готовый образ — Docker Hub (`<аккаунт>/tf-funnel:prod-<sha>`, workflow [deploy-funnel-prod.yml](#)). Или сборка **не на сервере стенда** ([P-49](#)):

```
docker build --platform linux/amd64 -f docs/backend/funnel/Dockerfile -t tf-funnel:prod docs/backend
```

Запуск (пара Vault — в переменных окружения сессии):

```
docker run -d --name tf-funnel --network think-fast-net --restart unless-stopped --memory 512m -v tf-funnel-data:/data -e TF_ENV=prod -e VAULT_ADDR=http://vault:8200 -e TF_VAULT_WAIT=900 -e TF_REDIS_URL=redis://tf-redis:6379/0 -e TF_AUTH_JWKS=http://tf-auth:8080/.well-known/jwks -e TF_FUNNEL_ARCHIVE_DAYS=14 -e TF_AUDIT_SPOOL=/data/audit.spool -e TF_FUNNEL_WS_ORIGINS=https://<домен> -e VAULT_ROLE_ID -e VAULT_SECRET_ID tf-funnel:prod
```

На prod **не задавать** `TF_FUNNEL_PULL`: это режим эмулятора.

A7. tf-model

Образ собирается из Think-Faster, пакет модели добавляется вторым слоем ([документация.md](#), [прил. Б.3–Б.4](#)):

```
docker build -f ML/service/Dockerfile -t tf-model:base .
```

```
printf 'FROM tf-model:base\nCOPY --chown=10001 . /models/current\n' | docker build -t tf-model:prod -f - <папка пакета модели>
```

```
docker run -d --name tf-model --network think-fast-net --restart unless-stopped --memory 2500m -v
tf-ml-work:/work -e TF_ENV=prod -e VAULT_ADDR=http://vault:8200 -e TF_VAULT_WAIT=900 -e
TF_REDIS_URL=redis://tf-redis:6379/0 -e TF_AUTH_JWKS=http://tf-auth:8080/.well-known/jwks -e
TF_MODEL_CLOCK=live -e TF_MEMORY=1GB -e VAULT_ROLE_ID -e VAULT_SECRET_ID tf-model:prod
```

Проверка: `https://<домен>/api/ml/health` → `ok: true`. Первый прогноз — после ближайшей границы часа + 120 с.

A8. tf-audit

Сначала схема `audit` (однократно, [SYSTEM.md VIII-3](#)):

1. строка `audit` в `postgree/db/schemas.conf` и два пароля в `secrets.conf`;
2. `scripts/secrets.sh init postgree`, выкатка `postgree`;
3. `psql -U audit_admin -d tf -f docs/backend/audit/schema.sql`.

Затем образ `docs/backend/audit/Dockerfile` (контекст `docs/backend`) и запуск с `TF_AUDIT_DB=host=tf-postgres dbname=tf user=audit_user` (или шаг `audit` в [deploy-apps-prod.yml](#)).

A9. Почта и Telegram

```
TF_STAND=prod scripts/deploy.sh mailing && TF_STAND=prod scripts/deploy.sh telegram
```

Если хостинг закрывает исходящие 25/465/587 — SMTP-релей на порту 2587 ([mailing/README.md](#)).

A10. Источник данных

- **Шина:** `sub` её учётки → `TF_FUNNEL_SERVICE_SUBS`, `IP` → `FUNNEL_EVENTS_ALLOW`, выкатка `web-server` ([SYSTEM.md 9.3](#)).
- **Эмулятор** (только не на `prod`): workflow [deploy-emulator-dev.yml](#) и `TF_FUNNEL_PULL=http://tf-emulator:8000` у воронки.
- **История:** пакеты из CSV в `POST /api/funnel/events` ([04. Методы 4.6](#)).

A11. Проверка

Проверка	Ожидается
<code>docker ps -a --format '{{.Names}}\t{{.Status}}'</code>	всё Up (healthy), <code>*-init</code> — Exited (0)
<code>https://<домен>/</code>	страница входа
<code>/api/auth/health</code> , <code>/api/bff/health/ready</code> , <code>/api/funnel/health</code> , <code>/api/ml/health</code>	200
вход под администратором	видны все окна
<code>docker logs tf-mail / tf-tg</code>	вход ... — ok, RabbitMQ: читаю ...

6.3. Вариант Б: локальный стенд потока

Файлы — [Think-Faster: docs/backend/stand.yml](#) и [compose.yml](#). Redis, RabbitMQ и Kafka берутся из клона `think-infra` без изменений. К ним добавляются dev-Vault с тестовыми секретами, PostgreSQL со схемой

аудита и эмулятор.

```
git clone https://github.com/GroznyiBombila/Think-Faster.git && git clone https://github.com/Think-Faster/think-infra.git
```

```
cd Think-Faster && cp docs/backend/stand.env.example docs/backend/stand.env
```

В `stand.env` указать `TF_INFRA`=<путь к `think-infra`>, `TF_TEST`=<путь к `think-test`>, `TF_MODEL_BUNDLE_DIR`=<папка пакета модели> и тестовые пароли.

```
docker network create think-fast-net
```

```
docker compose -f docs/backend/stand.yml --env-file docs/backend/stand.env up -d
```

```
docker compose -f docs/backend/compose.yml --env-file docs/backend/stand.env --env-file docs/backend/.vault/approle.env up -d --build
```

Цепочка: эмулятор → воронка (`TF_FUNNEL_PULL`) → `tf.ingest.*` → модель → `tf.forecast.results`. Проверка — `/status` воронки и модели, события в `audit.events`. Панель эмулятора открывается на `http://127.0.0.1:8090`: ускорение времени, ручные значения, сбои датчиков.

Особенности:

- `dev-Vault` держит секреты в памяти, после его перезапуска — снова `up tf-vault-seed`;
- без открытого ключа `tf-auth` ручки с токеном отвечают `503`, а `/health` и поток работают.

6.4. Вариант В: обновление развёрнутого стенда

Что	Как
Инфраструктура	пуш в ветку стенда (<code>dev / prod</code>) <code>think-infra</code> → <code>workflow deploy-<сервис>.yml</code> ; вручную — <code>TF_STAND=<стенд> scripts/deploy.sh <сервис></code> (SYSTEM.md 4.3)
<code>tf-auth</code> , <code>tf-bff</code> , <code>tf-front</code>	пуш в <code>prod</code> репозитория → <code>deploy-prod.yml</code> ; или шаги <code>auth bff</code> в <code>deploy-apps-prod.yml</code> <code>think-infra</code>
<code>tf-funnel</code> , <code>tf-model</code>	Actions → <code>deploy-funnel-prod.yml</code> / <code>deploy-model-prod.yml</code> в <code>Think-Faster</code>
После перезагрузки сервера	распечатать Vault (SYSTEM.md 4.5)
Откат	прежний тег образа (воронка); <code><сервис>.old</code> в <code>/srv/thinkfaster/services</code> (<code>tf-auth</code> , <code>tf-bff</code> , <code>tf-front</code>); <code>git checkout <коммит> + deploy.sh</code> (инфраструктура)

6.5. Частые проблемы при развёртывании

Симптом	Причина	Решение
502 на <code>/api/*</code> после перезагрузки	Vault запечатан	распечатать двумя ключами
Сертификат не выпускается	нет A-записи или закрыт порт 80	проверить DNS и порт; <code>--skip web-server</code> до готовности DNS

Симптом	Причина	Решение
Сервер перестал отвечать во время выкатки	сборка образа на сервере съела память	собирать в CI или локально
<code>invalid role or secret ID</code>	пара Vault другого стенда или роли	выдать заново <code>service-credentials</code> без <code>--rotate</code>
Письма не уходят, <code>Network is unreachable</code>	хостинг режет почтовые порты	релей на 2587
<code>Waiting for a runner</code> в GitHub	нет self-hosted runner с меткой стенда	подключить runner или выкатывать вручную

Полный список — [SYSTEM.md, Часть VI](#).